

Estructura y Uso de Genéricos en Java

Los genéricos transforman la forma en que gestionamos los datos, pasando de un modelo basado en "objetos genéricos" a uno de "tipado fuerte" que el compilador puede verificar.

1. Sintaxis para Definir una Clase Genérica

Para crear una clase genérica, añadimos el parámetro de tipo entre los símbolos < > justo después del nombre de la clase.

Estructura base:

Java

```
// Definimos 'T' como un marcador de posición para el tipo futuro
public class Caja<T> {
    private T contenido; // Atributo del tipo genérico [cite: 11]

    public void guardar(T objeto) {
        this.contenido = objeto;
    }

    public T obtener() {
        return contenido;
    }
}
```

Nomenclatura Estándar

Aunque podrías usar cualquier letra, por convención profesional usamos:

- **T**: Tipo general (Type).
- **E**: Elemento (utilizado en colecciones).
- **K / V**: Clave y Valor (Key / Value).
- **N**: Número (Number).

2. Cómo Instanciar y "Llamar" a una Clase Genérica

Cuando creamos un objeto de nuestra clase genérica, debemos especificar el tipo concreto que queremos usar.

Ejemplo de Uso:

Java

```
public class Main {
    public static void main(String[] args) {
        // Caja para guardar un String
        Caja<String> cajaDeTexto = new Caja<>();
        cajaDeTexto.guardar("Hola Mundo");

        // Caja para guardar un número entero
        Caja<Integer> cajaNumerica = new Caja<>();
        cajaNumerica.guardar(123);

        System.out.println(cajaDeTexto.obtener()); // Devuelve un
String directamente [cite: 14]
    }
}
```

Nota de seguridad: Intentar guardar un número en `cajaDeTexto` generará un error de compilación, evitando fallos inesperados en el futuro.

3. Ejemplo con Múltiples Parámetros: Clase `Par<K, V>`

A veces necesitamos relacionar dos tipos distintos. Aquí es donde los genéricos múltiples brillan.

Java

```
public class Par<K, V> {
    private K clave;
    private V valor;

    public Par(K clave, V valor) { // Constructor que recibe ambos
tipos [cite: 62]
        this.clave = clave;
        this.valor = valor;
    }

    public void mostrar() {
        System.out.println("Clave: " + clave + " | Valor: " + valor);
    }
}
```

Casos de uso reales:

- **Usuarios:** `Par<String, Integer> p1 = new Par<>("Edad de Juan", 25);`.

- **Geografía:** `Par<String, String> p2 = new Par<>("Argentina", "Buenos Aires");`.
- **Inventario:** `Par<Integer, Boolean> p3 = new Par<>(101, true);` (ID y si está activo).

4. El Rey de las Colecciones: `ArrayList<E>`

Antes de Java 5, las listas no sabían qué contenían y era obligatorio hacer "casteos" (conversiones manuales) constantes.

Comparativa: Antes vs. Después de los Genéricos

Característica	Forma Antigua (Sin Genéricos)	Forma Moderna (Con Genéricos)
Declaración	<code>ArrayList lista = new ArrayList();</code> +2	<code>ArrayList<String> lista = new ArrayList<>();</code>
Seguridad	Podías meter un <code>Integer</code> en una lista de <code>String</code> por error.	El compilador bloquea tipos incorrectos. +
Casteo	<code>String s = (String) lista.get(0);</code> (Obligatorio)	<code>String s = lista.get(0);</code> (Automático)

Código de ejemplo con `ArrayList`:

```
Java
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");
nombres.add("Luis");

// Java sabe que cada elemento es un String, no necesitas castear.
String primerNombre = nombres.get(0);
```

5. El Concepto de "Eliminación de Tipos" (Type Erasure)

Es fundamental que tus alumnos sepan que los genéricos son "ayudas" para el programador y el compilador. Una vez que el programa se ejecuta, Java borra la información del tipo genérico para mantener la compatibilidad.

- **¿Qué significa?:** En tiempo de ejecución, una `Caja<String>` y una `Caja<Integer>` se ven igual para la Máquina Virtual de Java.
- **Consecuencia:** Por esto mismo no se pueden crear arreglos de tipos genéricos (`new T[10]`) ni usar `instanceof` directamente con el parámetro de tipo.